

त्रिभूमि Tribhoomi

3D ULPIN Generation and Vertical Property Mapping System, with Property Identity and Integrity

Smart India Hackathon 2026 · Problem Statement SIH26011 Ministry of Rural Development · Department of Land Resources

Live prototype: <https://tribhoomi.onrender.com> · Source: <https://github.com/kkrrishagarwal/tribhoomi>

1. The problem in one paragraph

India's land records identify a *parcel* of land with a 14-digit ULPIN. That number describes a flat outline on the ground. It says nothing about what is stacked above or below it: the twelve floors of a tower, the flat on the eighth floor, the basement parking, the metro tunnel underneath, or the skywalk over it. As cities grow vertically, ownership disputes increasingly happen *inside* buildings: two flats surveyed into the same space, a builder quietly renumbering or shrinking a unit after selling it, a basement dug into a utility corridor. Today there is no shared, verifiable record of these vertical units, and a buyer has no independent way to check one before paying.

2. What Tribhoomi does

Tribhoomi extends the parcel record upward and downward and wraps it in a lifecycle that makes every change traceable.

	BUILDER	create project → add building → add unit → draw boundary → automatic validation → submit
	AUTHORITY	review evidence → approve / request changes / reject (every decision is audited)
	IDENTITY	Tribhoomi Property ID · Property Passport · QR · integrity score
	INVESTOR	search → verify before you invest → history → before/after → evidence report
	OWNER	my property → change alerts → approve/reject modifications → disputes

The one-line promise: **Tribhoomi does not just register a property. It keeps a traceable spatial history of that property and helps the authority detect and understand changes before they become boundary or ownership disputes.**

3. Functionality, feature by feature

3.1 Hierarchical 3D identity

Every parcel, building, level, unit and sub-surface layer gets its own identifier, generated by a pure, tested engine.

- **Technical land-record ID (ULPIN-format, prototype).** SS-DDD-VVVV-PPPPP-Bnn-Fnn-Unn-L. The first four segments are the existing 14-digit 2D ULPIN, so any unit ID truncates back to its parcel. Bnn building, Fnn/Lnn level above or below ground, Unn unit, and a layer letter: S surface, A above ground, G underground, R air-rights. Zero in any vertical segment means "the whole thing one level up", which keeps every ID unique and lets a parent be derived by truncation.
- **Tribhoomi Property ID (TPID)**, for example TRB-TA1-F08-U03: a readable identity for people and QR codes, linking project, building, floor and unit. It is Tribhoomi's own identifier and is never presented as an official ULPIN.

3.2 Builder workflow

- Create a project (land parcel), add buildings with floors, and register units floor by floor.
- **Visual boundary editor:** the builder draws a unit on the floor plan by dragging; corners and edges resize; live area in m² and sq ft. No coordinates are ever typed.
- **Automatic validation** in plain language with a score: valid shape, unique unit number, declared versus drawn area, inside the building, inside the parcel, no overlap with neighbours. Conflicts are explained with "why this matters".
- **Pre-submission checklist** stops bad submissions before they reach the authority.
- Submit for verification. A builder cannot mark their own unit verified.
- **Modification requests** reuse the same editor. Verified geometry is never overwritten; a change becomes a new version only after approval.

3.3 Authority workflow

- **Command center** with pending registrations, conflicts, modification requests, disputes and a **review-priority queue**.
- **Review page** with evidence: the unit's geometry and its neighbours, parent building and parcel, validation results, previous versions, submission and audit history.
- **What changed?** compares the registered version with the proposed one: area before and after, percentage change, which boundary moved and by how much, the changed region, containment changes, and any new overlap with a named neighbour.
- **Simulate modification:** the officer can drag the boundary to test a scenario and see area, overlap, validation and priority change, under a clear "SIMULATION, NOT SAVED" badge.
- **Structured decisions:** approve, request changes or reject, with a reason category and note stored in the immutable audit trail. Approval issues a verification ID.
- **Spatial conflict map** of every stored unit footprint coloured by its stored status.

3.4 Investor and owner workflow

- **Discover:** search by project, building, unit number or TPID; every result shows its verification status.
- **Property page** with a **Trust Summary** (verification, geometry, history, changes, disputes, data completeness) and an overall status: currently consistent, review recommended, or potential conflict detected.
- **Verify Before You Invest:** seven checks with a plain result.

- **Property Passport** with a QR code that opens a public page answering three questions: Is it registered? Is the current version verified? Is there a current spatial conflict? No owner data is ever shown publicly.
- **Property risk timeline**: when the property started needing attention, derived only from stored versions, proposals, disputes and audit events.
- **Watch this property** alerts, dispute filing, and a downloadable **evidence package** PDF.

3.5 Integrity engine underneath

- Registration or verification locks a baseline (version 1). Versions are append-only and never edited or deleted.
- A silent change therefore shows up as an unapproved version with a warning banner, on the risk timeline, and in the priority queue.
- An audit event records who did what, when, before and after, for every project, building, unit, submission, decision, modification, watch and dispute.
- A **topology validator** detects unit overlaps, encroachments beyond the footprint or parcel, and basement clashes with metro or utility corridors.
- A **review-priority heuristic** ranks work for the authority with every contributing factor listed under "Why this priority?". It is labelled as a Tribhoomi heuristic, not an official risk score.

3.6 Geospatial and AI layer

- 2D map with the parcels and buildings (Leaflet, OpenStreetMap).
- Optional 3D globe with real terrain and OSM Buildings (CesiumJS), our buildings drawn one slab per floor.
- Three.js unit viewer: click any unit, explode floors, see conflicts in red, underground layers as translucent volumes.
- One real AI inference pass: a pretrained segmentation model (SegFormer, trained on ADE20K) extracts building footprints from an aerial tile of Noida Sector 18 and vectorises them.
- A **market-context layer** of real NCR projects with RERA IDs, shown with a per-row confidence label and kept strictly separate from all registry logic, so no real company is ever associated with a demo dispute.

4. How it is built

Layer	Technology	Why
Frontend	Next.js 14 (React, TypeScript), Tailwind CSS	Fast pages, typed API client, one codebase for all roles
Maps and 3D	Leaflet + OpenStreetMap, CesiumJS with Cesium ion, Three.js via react-three-fiber	Real basemaps and terrain, per-unit 3D interaction
Backend	FastAPI (Python 3.12), SQLAlchemy ORM, Pydantic	Same language as the ML model, strict request validation

Layer	Technology	Why
Geometry	Shapely, own bounding-volume topology checks	Deterministic overlap, containment and displacement maths
Database	SQLite by default, PostgreSQL + PostGIS via one setting	Zero-setup demo; production spatial database ready
AI	Hugging Face Transformers, PyTorch (CPU)	Pretrained model, no training required
Documents	jsPDF, qrcode	Evidence package and passport QR in the browser
Deployment	Docker, Render (free tier), Hugging Face Spaces option	One container, one URL
Tests	pytest, FastAPI TestClient, Puppeteer-driven browser checks	40 automated tests plus end-to-end UI runs

Coding approach. The core logic lives in small, pure Python modules with no framework dependency: the ID engine, the topology validator, the plain-language validation, and the analysis layer (geometry change, impacted neighbours, risk timeline, priority, completeness, trust summary, simulation). Each is unit-tested before it is wired to an API or a screen. Routers are thin. The frontend consumes one typed API client. All derived scores show their method. Every analytical output is labelled as a stored record, Tribhoomi analysis, or demonstration data.

Security model in the prototype. Roles (builder, investor, owner, authority) are simulated in Demo Mode; the app says so on every screen. Even so, permissions are enforced on the server: a builder cannot approve, an investor cannot edit, verified geometry cannot be overwritten, and public pages never return owner details. Replacing the demo identity with real authentication means swapping one dependency.

5. Who uses it

User	What they get
Builders and developers	A clean way to register towers and units, immediate validation, fewer rejected submissions, a verifiable identity for each unit that helps sales
Home buyers and investors	A search, a QR scan and a plain verdict before paying; history and evidence they can download
Property owners	A locked baseline for their unit, alerts on any proposed change, approval rights, and a dispute channel
Land-records and planning authorities	A prioritised review queue, evidence-based decisions, simulation, an audit trail, and a spatial view of conflicts

User	What they get
Utilities and transit agencies	Underground and air-rights corridors as first-class records, so basement or foundation clashes are detected
Courts, banks and insurers	A traceable version history and evidence package for a specific unit

6. Feasibility

Technical. Everything in the prototype runs today on free infrastructure with a single container. The identity scheme is a strict superset of the current ULPIN, so adoption does not require changing existing parcel numbers. Geometry is stored as GeoJSON with Shapely maths, and the same models run on PostGIS by changing one connection string. The heavy parts, 3D and AI, are optional and lazy-loaded; the core registry pages work on any device without a GPU.

Operational. The workflow mirrors how registration already happens: builders submit, an authority approves, and records become public. Tribhoomi adds validation before submission and evidence during review, which shortens the authority's work instead of adding to it. Data entry needs no GIS skill: builders draw on a floor plan and never type coordinates.

Data. Real deployment would connect three inputs that already exist: parcel geometry from state land-record systems, building plans and floor layouts from sanction records, and RERA project registrations. Height data can come from drone or LiDAR surveys where available, or from sanctioned floor heights initially. The prototype simulates these inputs with a synthetic dataset and states so on every screen.

Economic. The stack is open source. Hosting cost for a district is a few hundred rupees per month at prototype scale and scales linearly. The value is in avoided disputes: a single prevented boundary litigation covers years of running cost.

Legal and trust. Tribhoomi never claims to replace the official land record or to verify legal ownership. It provides a verifiable digital identity and history that an official record can reference. Every score explains its method; every analytical statement uses "potential conflict" or "requires review" rather than accusatory language.

7. What is real and what is simulated

Real: the ID engine, geometry validation, conflict detection, versioning, audit trail, decision workflow, analysis layer, maps, terrain, the AI inference pass, the RERA market context, and all tests.

Simulated: authentication (Demo Mode), the property dataset (ten NCR projects, around 340 units, with staged stories), image georeferencing for the AI page, and the official land-record linkage, which is reported as missing in the data-completeness check rather than faked.

8. Roadmap after the hackathon

1. Real authentication with role provisioning for authority officers, and builder onboarding tied to RERA registration.

2. PostGIS deployment with polygon units (not only rectangles), so irregular flats validate exactly.
3. Import of sanctioned plans and drone or LiDAR heights to populate floors and volumes.
4. Digital signatures on verification decisions and a public verification API for banks and registrars.
5. Integration hooks for state land-record systems so the official 2D ULPIN links to each vertical identity.

9. Two-minute demo

1. Open the guided demo and click "Create scenario". Units 3-B, 3-C, 3-D are registered and verified; 3-C's builder proposes a 1.5 m extension that overlaps 3-D.
 2. Open 3-C: trust summary says review recommended, priority is critical with the reasons listed, "What changed?" shows +13.6% area, east boundary moved 1.5 m, overlap of 8% with 3-D.
 3. As the authority, the unit is at the top of the priority queue. Open the review, try "Simulate modification", then request a correction with the reason "Neighbour conflict".
 4. As the builder, resubmit a 0.4 m extension. As the authority, approve it. Version 2 is recorded.
 5. Scan the passport QR with a phone: registered yes, verified yes, no conflict. No private data shown.
-

Prototype built for SIH26011. All properties, builders and owners are a demonstration dataset. The Tribhoomi Property ID and Tribhoomi scores are internal to Tribhoomi and not official government values.